



Sistemas Distribuidos

Profesor: Oscar Manuel Ruiz Hurtado

Evidencia S11

Una verdad, varias copias

Sierra García Mariana

Grupo: 02

Facultad de Ingeniería

Bitácora de Laboratorio:

1. Parte 0

Antes de implementar la estrategia de caché se midió el comportamiento del sistema solicitando el saldo de la cuenta de Ana diez veces consecutivas y cada una de las consultas impactó directamente a la base de datos transaccional resultando en al menos 10 lecturas registradas en el log del servicio cuenta

Las estadísticas del servidor Redis confirman que la caché aún no interviene en el flujo de consulta mostrando los contadores en ceros (`keyspace_hits:0` y `keyspace_misses:0`).



Figura 1: Mas de 10 consultas

2. Parte 1

Tras configurar las abstracciones de Spring Boot en el microservicio cuenta se repitió la prueba solicitando el saldo de la cuenta diez veces consecutivas

Se comprobó que solo la primera petición impactó a PostgreSQL mientras que las nueve restantes fueron respondidas instantáneamente desde Redis, se evidencia al observar que la marca de tiempo en el JSON es idéntica en todas las respuestas y el contador de lecturas se mantuvo en 1



Figura 2: Lectura directa

Para validar la invalidación de la memoria se realizó un abono de prueba a la misma cuenta y al solicitar el saldo nuevamente el registro de lecturas a Postgres incrementó a 2 demostrando que @CacheEvict eliminó correctamente el saldo obsoleto en Redis y forzó al sistema a recuperar el dato transaccional actualizado



Figura 3: Lectura directa, saldo actualizado

3. Parte 2

Para desacoplar la base de datos transaccional de los datos de solo lectura se migró el almacenamiento de notificaciones de PostgreSQL a MongoDB esto permitio guardar documentos polimórficos donde cada aviso tiene una estructura distinta según su canal sin necesidad de esquemas rígidos o tablas dispersas

Al consultar la colección avisos directamente en la base de datos se observan las distintas formas de los documentos conviviendo en el mismo espacio

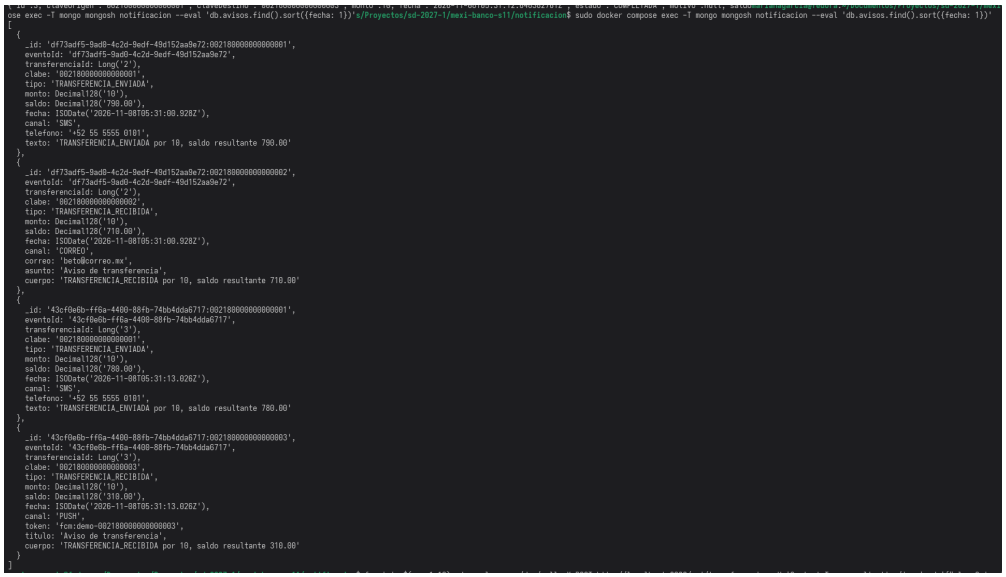


Figura 4: Diferentes formas de documentos

Para validar el endpoint de lectura, se generaron múltiples transferencias y se consultó la bitácora de la cuenta destino mediante el API Gateway. El sistema limitó exitosamente la respuesta a los 10 avisos más recientes, ordenados descendentemente por fecha.



Figura 5: Endpoint GET regresando un límite estricto de 10 avisos recientes

Se comprobó la eficiencia de la lectura mediante un análisis de ejecución la consulta optimizada utilizó el índice compuesto clabe_fecha ejecutando una etapa IXSCAN que examinó exactamente 10 documentos. Al forzar una consulta sin índice (COLLSCAN) el motor tuvo que examinar la totalidad de la colección en memoria para resolver la misma petición.

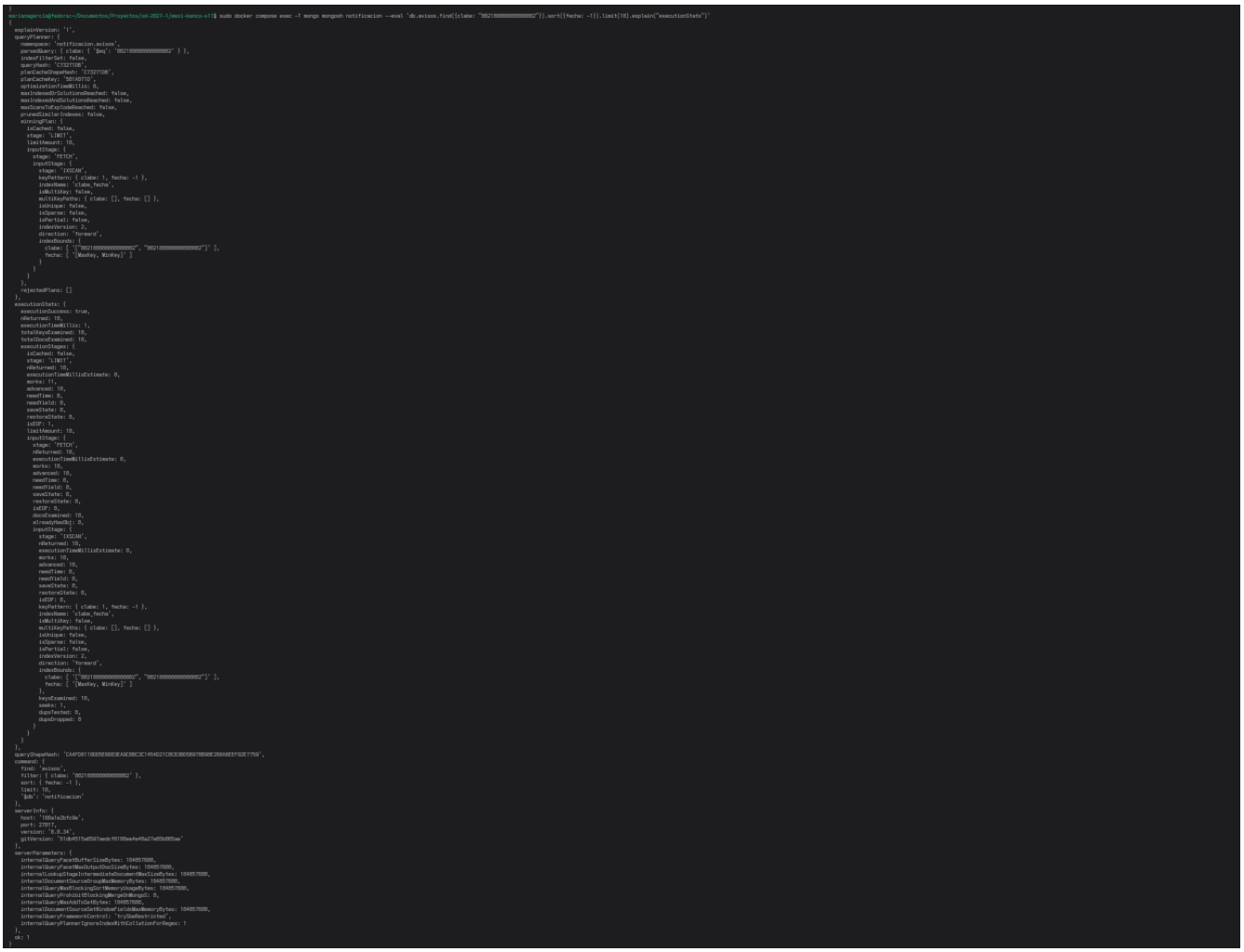


Figura 6: IXSCAN



Figura 7: COLLSCAN

4. Parte 3

Se validó esta restricción inyectando manualmente el mismo evento de transferencia dos veces directamente en el Exchange de RabbitMQ mediante su API HTTP

Al consultar MongoDB el contador de documentos para ese eventoId se mantuvo estrictamente en 2 confirmando que no hubo duplicidad, al revisar las bitácoras del contenedor notificacion se comprobó que el bloque de control de excepciones atrapó el error de llave duplicada proveniente de la base de datos y lo manejó correctamente registrando la advertencia en el log

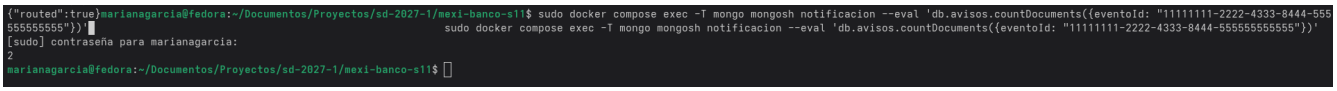


Figura 8: avisos: origen y destino

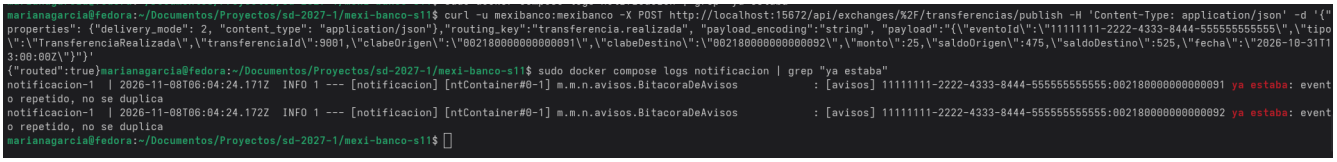


Figura 9: Evento repetido

5. Parte 4

Se demostró que la caché en Redis almacena una copia temporal y no constituye la fuente de la verdad transaccional, al modificar el saldo directamente en PostgreSQL el mecanismo de invalidación @CacheEvict no se activó

Al consultar el saldo mediante el API Gateway mientras la copia seguía vigente en Redis el sistema continuó mostrando el saldo obsoleto durante varios segundos, hasta que el TTL expiró y el servicio volvió a consultar la base de datos principal.

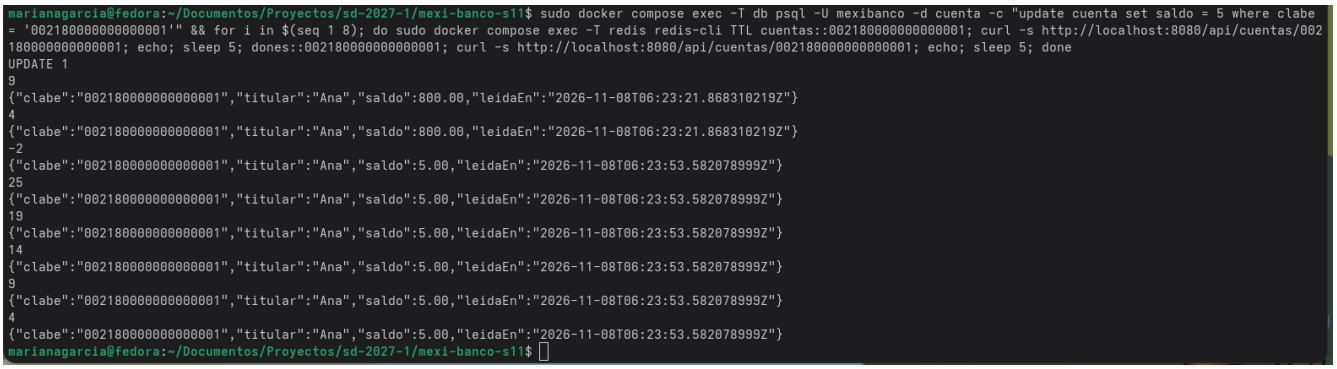


Figura 10: El microservicio muestra un saldo desactualizado

6. Preguntas de Cierre

1. **¿Por qué MongoDB para los avisos y no para el saldo?** Porque los avisos tienen formas distintas según el canal se escriben una vez y no se modifican, e saldo maneja dinero y exige consistencia estricta que solo una base relacional como PostgreSQL puede garantizar

2. **¿Qué hit ratio mediste en A y en B, y por qué cambia?** En A fue de 99 % por consultas directas en Redis y en B baja porque las transferencias borran el caché provocando que las lecturas fallen y vayan a la base de datos, el caché aporta velocidad masiva.

3. **¿Qué pasaría si la autorización del cargo leyera el saldo de la caché?** El sistema podría autorizar pagos con dinero obsoleto si hubo cambios por fuera de la aplicación rompiendo la integridad financiera

4. **Al aplicar un movimiento, ¿por qué se hace DEL y no SET? ¿Y por qué después del commit?** Se hace DEL para evitar conflictos de concurrencia y dejar que la siguiente lectura vuelva a cargar el dato, debe ser después del commit para evitar vaciar la caché antes de que Postgres guarde los cambios reales previniendo lecturas con datos viejos